НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт з виконання комп'ютерного практикуму №2
з дисципліни «Спеціальні розділи обчислювальної математики»

БАГАТОРОЗРЯДНА МОДУЛЯРНА АРИФМЕТИКА

Виконав студент
групи ФІ-43
Деркач Трохим Тарасович

Київ — 2026

Зміст

1 Мета роботи **2**

2 Завдання до комп'ютерного практикуму **2**

3 Теоретичні відомості **2**
3.1 Подання чисел 2
3.2 НСД та НСК 2
3.3 Додавання та віднімання за модулем 3
3.4 Редукція Барретта 3
3.5 Піднесення до степеня за модулем 3
3.6 Додаткове завдання: ефективне піднесення до квадрату 3

4 Результати роботи програми **4**
4.1 Операції над 1024-бітними числами 4
4.2 Контроль коректності (пункт Б) 4
4.3 Час виконання операцій (пункт В) 5
4.4 Порівняння ефективності піднесення до квадрату (додаткове завдання) . 6

5 Висновки **6**

6 Додаток. Лістинг коду програми **8**
6.1 Файл bignumarifmetic.h 8
6.2 Файл bignumarifmetic.cpp 8
6.3 Файл tests.h 13
6.4 Файл tests.cpp 13
6.5 Файл benchmarks.h 16
6.6 Файл benchmarks.cpp 16
6.7 Файл main.cpp 18

1 Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Завдання до комп'ютерного практикуму

Пункт А. Доопрацювати бібліотеку для роботи з m-бітними цілими числами, створену на комп'ютерному практикумі №1, додавши до неї такі операції:

- обчислення НСД та НСК двох чисел;
- додавання чисел за модулем;
- віднімання чисел за модулем;
- множення чисел та піднесення чисел до квадрату за модулем;
- піднесення числа до багаторозрядного степеня d за модулем n.

Модулярну арифметику реалізувати на базі редукції Барретта, піднесення до степеня — на базі схеми Горнера.

Додаткове завдання (варіант 1). Реалізація операції піднесення до квадрату (звичайного та модулярного) більш ефективним методом, ніж звичайним множенням; порівняння ефективності.

Пункт Б. Проконтролювати коректність реалізації алгоритмів, зокрема перевірити тотожності:

1. $(a+b)\cdot c \equiv c\cdot(a+b) \equiv a\cdot c + b\cdot c \pmod{n}$;
2. $n\cdot a \equiv \underbrace{a+a+\cdots+a}_{n} \pmod{m}$ для $n \geq 100$;
3. $a^{\varphi(n)} \equiv 1 \pmod{n}$ за умови $\gcd(a,n) = 1$ — для простого n та для $n = 3^k$, де $\varphi(3^k) = 2\cdot 3^{k-1}$;

розробити власні тести.

Пункт В. Обчислити середній час виконання реалізованих арифметичних операцій і кількість тактів процесора на кожну операцію; подати результати у вигляді таблиць.

3 Теоретичні відомості

3.1 Подання чисел

Бібліотека працює з беззнаковими числами сталої довжини в системі числення з основою $\beta = 2^{32}$: структура `BigNum` зберігає масив цифр типу `uint32_t`, наймолодша цифра лежить у комірці з індексом 0 (little-endian). Порівняно з практикумом №1 довжину числа збільшено з 2048 до 4096 біт (128 цифр). Причина — редукція Барретта: для модуля довжини k цифр добуток двох лишків має $2k$ цифр, а значення β^{2k} для обчислення μ — $2k+1$ цифру. Тому найбільший підтримуваний модуль має $k = 63$ цифри (2016 біт), а 1024-бітні модулі з умови задачі обробляються з запасом.

3.2 НСД та НСК

НСД обчислюється бінарним алгоритмом (алгоритмом Стейна), який використовує лише віднімання, порівняння та зсуви:

1. поки a і b обидва парні, ділимо їх на 2 і рахуємо кількість таких кроків k;
2. ділимо a на 2, доки воно парне;

3. поки $b \neq 0$: ділимо b на 2, доки воно парне; якщо $a > b$, міняємо їх місцями; $b := b - a$;
4. результат — $a \cdot 2^k$ (зсув a на k біт уліво).

На кожному кроці одне з чисел скорочується щонайменше на один біт, тому кількість ітерацій не перевищує $2\log_2 a$. НСК обчислюється за формулою $\operatorname{lcm}(a,b) = \dfrac{a}{\gcd(a,b)} \cdot b$; ділення виконується до множення, щоб проміжне значення не перевищувало a.

3.3 Додавання та віднімання за модулем

Обидві операції вимагають, щоб аргументи вже були зведені: $0 \le a, b < n$.

- **Додавання:** $c = a + b < 2n$; якщо $c \ge n$, то $c := c - n$. Оскільки модуль не довший за половину розрядної сітки, перенос із старшої цифри неможливий.
- **Віднімання:** $c = a - b$. Якщо $a < b$ (у тому числі у випадку $B > A$ з основного завдання), `LongSub` повертає позику, а в c лежить $a - b + 2^{4096}$; тоді $c := c + n$, і зайвий 2^{4096} зникає як перенос. У результаті отримуємо коректне значення $a - b + n \in [0, n)$.

3.4 Редукція Барретта

Нехай модуль n має k значущих цифр, а $x < n^2$ (не більше $2k$ цифр). Частка $q = \lfloor x/n \rfloor$ оцінюється величиною

$$\hat{q} = \left\lfloor \frac{\lfloor x/\beta^{k-1} \rfloor \cdot \mu}{\beta^{k+1}} \right\rfloor, \qquad \mu = \left\lfloor \frac{\beta^{2k}}{n} \right\rfloor,$$

де ділення на степені β — це відкидання молодших цифр (зсув), а μ залежить лише від n і обчислюється один раз (єдине справжнє ділення). Барретт довів, що $q - 2 \le \hat{q} \le q$, тому залишок $r = x - \hat{q}n$ приводиться до $[0, n)$ не більше ніж двома відніманнями n. Таким чином, замість ділення виконуються два множення, два зсуви та кілька віднімань.

Модулярне множення та піднесення до квадрату — це звичайне множення (піднесення до квадрату) з подальшою редукцією Барретта.

3.5 Піднесення до степеня за модулем

Використовується схема Горнера з проходом по бітах показника $B = b_{m-1}2^{m-1} + \cdots + b_0$ від молодшого до старшого:

```
C := 1
for i := 0 to m-1 do:
    if b[i] = 1 then C := Barrett(C * A)
    A := Barrett(A^2)
return C
```

Основа попередньо зводиться за модулем. На кожен біт показника припадає одне модулярне піднесення до квадрату та в середньому половина модулярного множення.

3.6 Додаткове завдання: ефективне піднесення до квадрату

Для числа $a = \sum_{i=0}^{n-1} a_i \beta^i$ маємо

$$a^2 = \sum_{i=0}^{n-1} a_i^2 \beta^{2i} + 2 \sum_{0 \le i < j < n} a_i a_j \beta^{i+j}.$$

Звичайне множення $a \cdot a$ обчислює всі n^2 добутків цифр, причому кожен добуток $a_i a_j$ при $i \neq j$ — двічі. Алгоритм піднесення до квадрату обчислює кожен попарний добуток лише один раз ($\frac{n(n-1)}{2}$ множень), подвоює суму зсувом на 1 біт і додає n діагональних квадратів a_i^2. Разом це $\frac{n(n+1)}{2}$ множень цифр замість n^2, тобто теоретичний виграш близько 2 разів. Модулярне піднесення до квадрату — `LongSquare` з подальшою редукцією Барретта; його використовує піднесення до степеня на кроці $A := A^2$.

4 Результати роботи програми

4.1 Операції над 1024-бітними числами

Для чисел A, B, N довжиною 1024 біти, причому $B > A$ (перевірка віднімання більшого числа від меншого):

```
A = CEBE60EFD949039FA8D8BAC7D3E17DB03F3E060531A329396FF9F9AB618600487E6EFA1059FA75C882F01AB7D0564B58030DFDF
6846DB25D7E02DD90B8F9C58350C7F5878858C561E5D2A665F3F51DA1F590C1150948B33DADE79A8F3868C7849AA74E2C2741705CFA
E461BC360850A4BF77DF2A6A8B0C50BD40FA4A2E73527E
B = D022E8F4FA56820673AE38CC8820C81662B720857E4FB620C7FCC6E3869E9435D24FE2B17F7A0769FCF5088580F8F7388CE84F8
151CF034FAEB7B9C9C5209C90FF0030191C71640436766C7D4A894774B05D2E9149278FA1C720BBF91D878E94DB602352D033997BD7
3D8E81D5DE0D3A3D43F4D0B4AC871C78A132618C1ACA9E
N = DCB6124729054A97AEE4A7EDFFC78672DE847485FB6DB32472C1B47F0741328CE0469FA2602985B97518B05112D9D65FF664F74
2B16DBB7C9E863514F0003E66D0B4BF8E9607AB378AA30DE8DC842D83A63C88CE01AB036763286EC63A60F30B052D78B79761D2B37F
AFAD485EEDA1D852AE3F0635840EA8A1BEF5D3BD9DD94D

gcd(A, B) = 2

lcm(A, B) = 540B70170FEFDBE51392A04A228B68856094F5BAD9D55B1C0102B00F1917D43A0B11F25CD410D19A40C03202817D565
038FAD4BEEB66977CDC96EBFE345612D09537DF2014476BBDD6CAEA5BB957FC540ACAA83653BA738A145DD7BD1902957D6BAD14477A
D812EF7D39517C0E140ED98690F5F51F3CA12B17836EF7D6A4B5A2B14641DA222B35978F04B8F7941582ADE074B8BB4C703E3FA3F63
84DE72E4D381C05C01AC781195C1B6F90463FCECD90D8BE695EE087737AB74906B7213777206C66EE6243CB2AD05E073796546661C0
76A170122934908F9E1D503B1FC395F2D16B339CCB85AE4F9CB9BCB286222D24A6107228AFF8106610CAD48079882AE2

A + B mod N = C22B379DAA9A3B0E6DA24BA65C3ABF53C370B204B4852C35C5350C0FE0E361F170783D1F794AF7790ACC72EC3E756
C309991563524CEFA308E3462458E1A23AD7F1366120EC27E2E91A604FA61FA3792FFB166D850C53F7811DFE7C21B8F630E70D9F8C7
60133725527242F5ACF8BC06AA0D94F4E9B384C4942336D7FCF043CF

A - B mod N = DB518A4207F7CC30E40F29E94B883C0CBB0B5A05AEC1263D1ABEE746E2289E9F8C65B7013AA9F417FB13C28362372
A7F6C8AA5B7E40C6A8A6DD158DBE3D96759227C84FD01EF0C9539FF47D185F003B0EB701B51C1CC270349EF4D5C55422BFAC474A390
EE6FA994A3568082BF17E542D4E2295FEB6293DCE65EBDBC5FF6612D

A * B mod N = 59A4F001DB99BB80EAFC4261DB01128CC5E599440EBDFFFF7D5813411D15A3AEFC1D4670F5B4CF77A10C8D5A5C66C
24D27EE2BD9A28F261557B5248E7388C4922B9E51CF2E9D426B9E50BF9E9D8C280B88434C70C2FDEC3199F589C00DB2F5D1B7C69AE8
D0AD0959B32210E819B2360B80224AC15DF996CB485E60C45DE06A9F

A ^ B mod N = 9F6F85C3D8955E1D06001A682881C7210D816D3F61FA2CB63EA65246FEFD2235F41212963CA2D37021741633E76F6
E245C6D17BCE78650585FBE558F964821990943788D1034ACF38D506F89CE0CBDD246E3A1504762F32AC10F78A16D2592005DBD977E
DDB5402857137CEF81EFD7B036CF201666B784BD630E5351903DD756
```

Усі результати збігаються зі значеннями, обчисленими незалежно (вбудованою довгою арифметикою Python).

4.2 Контроль коректності (пункт Б)

Тести 1–4 перевіряють тотожності з методички, тести 5–9 — власні. Кожен тест спочатку порівнює результат на фіксованих числах (модуль $n = 2^{255} - 19$) з еталонними значеннями, а потім перевіряє тотожність на сотнях–тисячах випадкових наборів із випадковими модулями довжиною до 2016 біт. Тотожність $a^{\varphi(n)} \equiv 1$ перевірено для простих $n = 2^{255} - 19$ та чисел Мерсенна $2^{127} - 1$, $2^{521} - 1$, $2^{607} - 1$, а також для $n = 3^k$, $k \leq 620$.

```
--- Running Tests (4096 bit numbers, modulus up to 2016 bit) ---

  n  = 7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFED
  mu = 20000000000000000000000000000000000000000000000000000000000000004C
  a  = 6C1A4F0E3B2D59877A13C4E2F05B6D18A9C3E7F2014B5D6E8F90A1B2C3D4E5F6
  b  = 3F2E1D0C9B8A79685746352413F2E1D0CFBEAD9C8B7A69584736251403F2E1D0
  c  = 1234567890ABCDEF1234567890ABCDEF

  (a+b)*c mod n   = CDCCA09184A1B5B1B90E37039BC68063AC88DFE8BB795659305430D1557A619
  c*(a+b) mod n   = CDCCA09184A1B5B1B90E37039BC68063AC88DFE8BB795659305430D1557A619
  a*c + b*c mod n = CDCCA09184A1B5B1B90E37039BC68063AC88DFE8BB795659305430D1557A619
Test 1 [(a+b)*c == c*(a+b) == a*c + b*c (mod n)]: Passed!
  100*a mod n       = 3A46E18F1DB6F8EBAFB8E8A5E3B69DA250869A8881707F30147F29D47F29DA54
  a + ... + a mod n = 3A46E18F1DB6F8EBAFB8E8A5E3B69DA250869A8881707F30147F29D47F29DA54
Test 2 [k*a == a+a+...+a (mod n), k = 100..1000]: Passed!
  a^(n-1) mod n, n = 2^255-19 = 1
Test 3 [a^phi(n) == 1 (mod n), n prime: 2^255-19, Mersenne 2^127-1, 2^521-1, 2^607-1]: Passed!
  n = 3^100              = 5A4653CA673768565B41F775D6947D55CF3813D1
  (a+1)^phi(n) mod 3^100 = 1
Test 4 [a^(2*3^(k-1)) == 1 (mod 3^k), a not divisible by 3]: Passed!
  x               = 2EB0FA96DAC3096575C2F62F9AC6A4DF644CE7ACD06204C975B7440EFA8AA3EBDC236081F0D0EE0D3A81A6
  D5BA2DC3187B4BC672C1E160A752F78142C05E7F57
  Barrett(x)       = 4A6892E669C4531CB57231E6B3AA3C415EB62A19B06E168ECC2B9B7BF0F2D556
  x mod n (DivMod) = 4A6892E669C4531CB57231E6B3AA3C415EB62A19B06E168ECC2B9B7BF0F2D556
Test 5 [BarrettReduction(x) == x mod n, x < n^2]: Passed!
  gcd(a, b) = 6
  lcm(a, b) = 47253132B2A40A502D805AE83B8FF272E9D0FE2264B1CE6B9D44CFE9FC3728E360F348133AB5F8E368E37EE8AF11C
  7687455B4EC0A9B60640C7706C15460250
Test 6 [a, b divisible by gcd, gcd(a/g, b/g) == 1, gcd * lcm == a * b]: Passed!
  a + b mod n = 2B486C1AD6B7D2EFD159FA07044E4EE97982958E8CC5C6C6D6C6C6C6C7C7C7D9
  a - b mod n = 2CEC32019FA2E01F22CD8FBEDC688B47DA053A5575D0F416485A7C9EBFE20426
  b - a mod n = 5313CDFE605D1FE0DD327041239774B825FAC5AA8A2F0BE9B7A58361401DFBC7
Test 7 [(a+b)-b == a, (a-b)+b == a, a-a == 0 (mod n), results < n]: Passed!
  a * b mod n = 3A2D3819A7A5D04BCFBA5F049269E9ACAF8249429EE001A305C1354EC9B61516
  a^2 mod n   = 48A031F31C028489B8944CE4F0A3CE196D47FC161F1867E73DEE4227380B1A41
  a^b mod n   = 41FA9242A62F71FCBD59F3368FFA8400A445268C9A885C66B47D8ECEDBD20AD8
Test 8 [a*b mod n == (a*b) % n, a^2 == a*a (mod n), a^2 == a*a]: Passed!
Test 9 [a^(e+f) == a^e * a^f, a^(e*f) == (a^e)^f (mod n), a^0 == 1, a^1 == a]: Passed!

All tests passed (failed: 0)
------------------------------
```

4.3 Час виконання операцій (пункт B)

Вимірювання виконано на процесорі AMD Ryzen 9 7945HX (компілятор MSVC, оптимізація /O2). Для кожного розміру модуля згенеровано 256 випадкових модулів та аргументів; показник степеня має ту ж довжину, що й модуль.

```
--- Running Benchmarks (4096 bit numbers, 256 random moduli per size) ---

Modular arithmetic
| Operation                          | Iterations |        ns / op |    cycles / op |
|------------------------------------|------------|----------------|----------------|
| LongGcd (256 bit)                  |      16000 |        53163.9 |         132658 |
| LongLcm (256 bit)                  |       2400 |        81587.0 |         203581 |
| LongAddMod (256 bit)               |    1000000 |           95.2 |            237 |
| LongSubMod (256 bit)               |    1000000 |           74.0 |            185 |
| BarrettMu (256 bit)                |       2400 |        30385.5 |          75820 |
| BarrettReduction (256 bit)         |      50000 |         2147.3 |           5358 |
| x mod n via LongDivMod (256 bit)   |       2400 |        29066.9 |          72529 |
| LongMulMod (256 bit)               |      50000 |         3140.3 |           7836 |
| LongPowerMod (256 bit)             |        160 |       991418.8 |        2473830 |
| LongGcd (512 bit)                  |       6000 |       105671.8 |         263678 |
| LongLcm (512 bit)                  |        900 |       160181.7 |         399691 |
| LongAddMod (512 bit)               |    1000000 |           95.3 |            238 |
| LongSubMod (512 bit)               |    1000000 |           73.7 |            184 |
| BarrettMu (512 bit)                |        900 |        59685.1 |         148930 |
| BarrettReduction (512 bit)         |      50000 |         4049.7 |          10105 |
| x mod n via LongDivMod (512 bit)   |        900 |        58390.6 |         145700 |
| LongMulMod (512 bit)               |      50000 |         5945.9 |          14836 |
| LongPowerMod (512 bit)             |         60 |      3705735.0 |        9246765 |
| LongGcd (2016 bit)                 |       2000 |       405226.4 |        1011144 |
```

```
| LongLcm (2016 bit)                  |        300 |       607951.3 |        1516992 |
| LongAddMod (2016 bit)               |    1000000 |           83.2 |            208 |
| LongSubMod (2016 bit)               |    1000000 |           75.8 |            189 |
| BarrettMu (2016 bit)                |        300 |       180709.0 |         450916 |
| BarrettReduction (2016 bit)         |      50000 |        14513.7 |          36215 |
| x mod n via LongDivMod (2016 bit)   |        300 |       180438.3 |         450239 |
| LongMulMod (2016 bit)               |      50000 |        21947.3 |          54764 |
| LongPowerMod (2016 bit)             |         20 |     54428100.0 |      135812126 |

Variant 1: squaring vs multiplication
| Operation                           | Iterations |       ns / op |    cycles / op |
|-------------------------------------|------------|---------------|----------------|
| LongMul(a, a) (256 bit)             |     100000 |         967.9 |           2415 |
| LongSquare(a) (256 bit)             |     100000 |         176.7 |            441 |
| LongMulMod(a, a) (256 bit)          |      50000 |        3178.9 |           7932 |
| LongSquareMod(a) (256 bit)          |      50000 |        2382.6 |           5945 |
| LongMul(a, a) (512 bit)             |     100000 |        1887.5 |           4710 |
| LongSquare(a) (512 bit)             |     100000 |         241.7 |            603 |
| LongMulMod(a, a) (512 bit)          |      50000 |        5895.5 |          14711 |
| LongSquareMod(a) (512 bit)          |      50000 |        4225.1 |          10543 |
| LongMul(a, a) (2016 bit)            |     100000 |        7090.2 |          17692 |
| LongSquare(a) (2016 bit)            |     100000 |        1423.9 |           3553 |
| LongMulMod(a, a) (2016 bit)         |      50000 |       21898.4 |          54642 |
| LongSquareMod(a) (2016 bit)         |      50000 |       16155.3 |          40312 |

LongPowerMod: exponent has the same bit length as the modulus
cycles = TSC (__rdtsc), counts at the nominal CPU frequency
------------------------------
```

4.4 Порівняння ефективності піднесення до квадрату (додаткове завдання)

	Звичайне, нс			За модулем, нс		
Довжина	$a \cdot a$	a^2	виграш	$a \cdot a \bmod n$	$a^2 \bmod n$	виграш
256 біт	967.9	176.7	5.5×	3178.9	2382.6	1.33×
512 біт	1887.5	241.7	7.8×	5895.5	4225.1	1.40×
2016 біт	7090.2	1423.9	5.0×	21898.4	16155.3	1.36×

Таблиця 1: Піднесення до квадрату порівняно з множенням числа на себе

5 Висновки

У ході роботи бібліотеку багаторозрядної арифметики з практикуму №1 доповнено модулярними операціями: обчисленням НСД (бінарним алгоритмом) та НСК, додаванням, відніманням, множенням і піднесенням до квадрату за модулем на основі редукції Барретта та піднесенням до степеня за модулем схемою Горнера. Для підтримки 1024-бітних модулів довжину числа збільшено до 4096 біт. Коректність підтверджено перевіркою тотожностей з методички (зокрема теореми Ейлера для простих модулів і для $n = 3^k$) та власних тестів на тисячах випадкових наборів, а також порівнянням з незалежно обчисленими значеннями — усі тести пройдено. Віднімання за модулем коректно обробляє випадок, коли від'ємник більший за зменшуване ($B > A$).

Результати вимірювань узгоджуються з теоретичною складністю алгоритмів:

- **Додавання та віднімання за модулем** ($\approx$ 75–95 нс) — один-два проходи по масиву цифр; оскільки цикли йдуть по всій довжині числа, час не залежить від довжини модуля.

- **Редукція Барретта** у $\approx$ 12–14 разів швидша за знаходження остачі діленням `LongDivMod` (для 2016-бітного модуля 14.5 мкс проти 180 мкс). Обчислення μ коштує саме одне таке ділення, тому передобчислення окуповується вже з другої редукції за тим самим модулем.
- **Модулярне множення** складається з множення та редукції (ще двох множень), тому коштує приблизно втричі більше за звичайне множення.
- **Піднесення до степеня за модулем** має складність $O(m \cdot T_{mul})$, де m — довжина показника. Оскільки час множення в реалізації пропорційний кількості цифр множника, при показнику тієї ж довжини, що й модуль, час зростає квадратично: подвоєння довжини (256→512 біт) збільшує час у 3.7 раза, а збільшення в 3.94 раза (512→2016 біт) — у 14.7 раза ($3.94^2 \approx 15.5$).
- **НСД** виконує $O(n)$ ітерацій, кожна з яких містить зсув і віднімання по всій довжині масиву, тому час росте лінійно з довжиною чисел (53, 106 та 405 мкс для 256, 512 та 2016 біт). НСК дорожчий на одне ділення та множення.
- **Піднесення до квадрату (додаткове завдання)** у 5–8 разів швидше за множення числа на себе. Теоретичний виграш від обчислення кожного попарного добутку лише один раз становить $\approx$ 2 рази; решта різниці пояснюється тим, що `LongSquare` накопичує добутки одразу в позиції $i + j$ і обмежує цикли реальною довжиною числа, тоді як `LongMul` для кожної цифри множника робить три повні проходи по масиву. У модулярному варіанті виграш становить лише ≈ 1.35 раза, бо більшу частину часу займає редукція Барретта, однакова в обох випадках. Саме тому в піднесенні до степеня для кроку $A := A^2$ використовується `LongSquareMod`.

Подальше прискорення можливе насамперед за рахунок множення: накопичення добутків $a_j b_i$ одразу в позиції $i + j$ та обмеження циклів реальною довжиною чисел пришвидшило б і множення, і редукцію Барретта, а отже, й усі модулярні операції. Для НСД доцільно знімати кілька нульових молодших бітів одним зсувом замість побітового ділення на 2.

6 Додаток. Лістинг коду програми

Повний код і історія змін доступні в репозиторії: `https://github.com/troshxa/SROM_Derkach`.

6.1 Файл bignumarifmetic.h

```h
#pragma once
#include <cstdint>
#include <string>

constexpr int W = 32;
constexpr int BITS = 4096; // модуль для Barrett до (BITS/2 - 32) біт, тобто 1024-бітний N вміщається
constexpr int WORDS = BITS / W; //w=32
constexpr uint64_t BASE = 1ULL << W;
constexpr uint64_t MASK = BASE -1;

struct BigNum {
    uint32_t d[WORDS];
};

uint32_t LongAdd(const BigNum& a, const BigNum& b, BigNum& c);
uint32_t LongSub(const BigNum& a, const BigNum& b, BigNum& c);
int LongCmp(const BigNum& a, const BigNum& b);
BigNum ToBigNum(uint64_t x);
uint32_t LongMulOneDigit(const BigNum& a, uint32_t b, BigNum& c);
BigNum LongShiftDigitsToHigh(const BigNum& a, int x);
BigNum LongShiftDigitsToLow(const BigNum& a, int x);
BigNum LongShiftBitsToHigh(const BigNum& a, int x);
BigNum LongShiftBitsToLow(const BigNum& a, int x);
void LongMul(const BigNum& a, const BigNum& b, BigNum& c);
void LongSquare(const BigNum& a, BigNum& b);
int BitLength(const BigNum& a);
bool LongDivMod(const BigNum& a, const BigNum& b, BigNum& q, BigNum& r);
void LongPower(const BigNum& a, const BigNum& b, BigNum& c);
bool FromHex(const std::string& s, BigNum& a);
std::string ToHex(const BigNum& a);
bool FromBin(const std::string& s, BigNum& a);
std::string ToBin(const BigNum& a);

//lab2
void LongDiv(const BigNum& a, const BigNum& b, BigNum& q);
BigNum LongGcd(const BigNum&a,const BigNum&b);
BigNum LongLcm(const BigNum& a, const BigNum &b);
void LongAddMod(const BigNum& a, const BigNum& b, const BigNum& n,BigNum& c);
void LongSubMod(const BigNum& a, const BigNum& b, const BigNum& n,BigNum& c);

BigNum BarrettMu(const BigNum & n);
BigNum BarrettReduction(const BigNum& a, const BigNum & n, const BigNum& mu);
void LongMulMod(const BigNum& a,const BigNum& b, const BigNum& n, const BigNum& mu, BigNum& c);
void LongSquareMod(const BigNum& a, const BigNum& n, const BigNum& mu,BigNum& c);
void LongPowerMod(const BigNum& a, const BigNum& b, const BigNum& n, const BigNum& mu, BigNum& c);
```

6.2 Файл bignumarifmetic.cpp

```cpp
#include <utility>
#include "bignumarifmetic.h"

uint32_t LongAdd(const BigNum& a, const BigNum& b, BigNum& c){
    uint32_t carry = 0;
    uint64_t temp = 0;
    for (int i = 0; i < WORDS; i++){
        temp = (uint64_t)a.d[i] +b.d[i] + carry;
        c.d[i] = temp & MASK;
        carry = temp >> W;
    }
    return carry;
}

```

```
uint32_t LongSub(const BigNum& a, const BigNum& b, BigNum& c){
    uint32_t borrow = 0;
    int64_t temp = 0;
    for(int i = 0; i <WORDS; i++){
        temp = (int64_t)a.d[i] - b.d[i] - borrow;
        if(temp>=0){
            c.d[i] = (uint32_t)temp;
            borrow = 0;
        }else{
            c.d[i] = (uint32_t)(BASE + temp);
            borrow = 1;
        }
    }
    return borrow;
}

int LongCmp(const BigNum& a, const BigNum& b){
    int i = WORDS-1;
    while(i >=0 && a.d[i] == b.d[i]){
        i-=1;
    }
    if(i==-1) return 0;
    else{
        if(a.d[i]>b.d[i]) return 1;
        else return -1;
    }

}

BigNum ToBigNum(uint64_t x){
    BigNum a ={};
    a.d[0] = x & MASK;
    a.d[1] = x >> W;
    return a;
}

uint32_t LongMulOneDigit(const BigNum& a, uint32_t b, BigNum& c){
    uint32_t carry =0;
    uint64_t temp = 0;
    for(int i =0; i < WORDS; i++){
        temp = (uint64_t)a.d[i]*b + carry;
        c.d[i] = temp & MASK;
        carry = temp >> W;
    }
    return carry;
}

BigNum LongShiftDigitsToHigh(const BigNum& a, int x){
    BigNum c = {};
    if (x >= WORDS) return c;
    for(int i = x; i < WORDS; i++){
        c.d[i] = a.d[i-x];
    }
    return c;
}

BigNum LongShiftDigitsToLow(const BigNum& a, int x){
    BigNum c = {};
    if (x >= WORDS) return c;
    for(int i = 0; i < WORDS-x; i++){
        c.d[i] = a.d[i+x];
    }
    return c;
}

BigNum LongShiftBitsToHigh(const BigNum& a, int x){
    int digits = x / W;
    int bits = x % W;
    BigNum c = LongShiftDigitsToHigh(a, digits);
    uint32_t tail = 0;
    uint64_t temp =0;
    for(int i =0; i <WORDS; i++){
        temp = (uint64_t)c.d[i] <<bits;
        c.d[i] = (temp & MASK) | tail;
        tail = temp >> W;
    }
```

```
    return c;
}

BigNum LongShiftBitsToLow(const BigNum& a, int x){
    int digits = x / W;
    int bits = x % W;
    BigNum c = LongShiftDigitsToLow(a, digits);
    uint32_t tail = 0;
    uint64_t temp =0;
    for(int i =WORDS-1; i >=0; i --){
        temp = ((uint64_t)c.d[i]<<W) >>bits;
        c.d[i] = (temp >> W) | tail;
        tail = temp & MASK;
    }
    return c;
}

void LongMul(const BigNum& a, const BigNum& b, BigNum& c){
    BigNum tempC = {};
    BigNum temp = {};
    for(int i =0; i < WORDS; i++){
        if (b.d[i] == 0) continue;
        LongMulOneDigit(a,b.d[i],temp);
        LongAdd(tempC, LongShiftDigitsToHigh(temp,i), tempC);
    }
    c = tempC;
}

void LongSquare(const BigNum& a, BigNum& b) {
    BigNum t = {};
    BigNum d = {};
    uint32_t carry;
    uint64_t temp = 0;
    int n = (BitLength(a) + W - 1) / W;

    for(int i =0; i <n; i++){
        if (a.d[i] == 0) continue;
        carry=0;
        for(int j = i + 1; j < n && i + j < WORDS; j++){
            temp = (uint64_t)t.d[i + j] + (uint64_t)a.d[i] * a.d[j] + carry;
            t.d[i + j] = temp & MASK;
            carry = temp >> W;
        }
        if (i + n < WORDS) t.d[i + n] = carry;
    }

    t = LongShiftBitsToHigh(t, 1);

    for(int i = 0; i < n && 2 * i < WORDS; i++){
        temp = (uint64_t)a.d[i] * a.d[i];
        d.d[2 * i] = temp & MASK;
        d.d[2 * i + 1] = temp >> W;
    }

    LongAdd(t, d, b);
}

int BitLength(const BigNum& a){

    for(int i = WORDS-1; i >=0; i--){
        if(a.d[i]!=0){
            uint32_t v = a.d[i];
            int bits = 0;
            while (v!=0){
                v = v>>1;
                bits++;
            }
            return i*W +bits;
        }
    }
    return 0;
}

bool LongDivMod(const BigNum& a,const BigNum& b,BigNum& q,BigNum& r){
    int k = BitLength(b);
    if (k ==0){
```

```
        q = {};
        r = {};
        return false;
    }
    int t ;
    int s;
    BigNum c;
    BigNum tempR = a;
    BigNum tempQ = {};
    while(LongCmp(tempR,b)!=-1){
        t = BitLength(tempR);
        c = LongShiftBitsToHigh(b, t-k);
        if (LongCmp(c,tempR)==1){
            t--;
            c = LongShiftBitsToHigh(b,t-k);
        }
        LongSub(tempR,c,tempR);
        s = t-k;
        tempQ.d[s/W] |= 1u << (s%W);
    }
    q = tempQ;
    r = tempR;
    return true;
}

void LongPower(const BigNum& a,const BigNum& b,BigNum& c){
    BigNum tempC = {};
    tempC.d[0] = 1;
    for(int i = BitLength(b)-1; i >=0; i--){
        if ((b.d[i / W] >> (i % W)) & 1){
            LongMul(tempC, a, tempC);
        }
        if(i!=0){
            LongSquare(tempC,tempC);
        }
    }
    c = tempC;
}

static int CharValue(char ch, int bitsPerChar){
    int v = -1;
    if (ch >= '0' && ch <= '9') v = ch - '0';
    else if (ch >= 'a' && ch <= 'f') v = ch - 'a' + 10;
    else if (ch >= 'A' && ch <= 'F') v = ch - 'A' + 10;
    if (v >= (1 << bitsPerChar)) return -1;
    return v;
}

static bool FromPow2String(const std::string& s, int bitsPerChar, BigNum& a){
    if (s.empty()) return false;
    BigNum t = {};
    int pos = 0;
    for (size_t k = s.size(); k > 0; k--){
        int v = CharValue(s[k - 1], bitsPerChar);
        if (v < 0) return false;
        if (pos >= BITS){
            if (v != 0) return false;
            continue;
        }
        t.d[pos / W] |= (uint32_t)v << (pos % W);
        pos += bitsPerChar;
    }
    a = t;
    return true;
}

static std::string ToPow2String(const BigNum& a, int bitsPerChar){
    const char* digits = "0123456789ABCDEF";
    int n = BitLength(a);
    if (n == 0) return "0";
    int chars = (n + bitsPerChar - 1) / bitsPerChar;
    uint32_t mask = (1u << bitsPerChar) - 1;
    std::string s;
    for (int pos = (chars - 1) * bitsPerChar; pos >= 0; pos -= bitsPerChar){
        s += digits[(a.d[pos / W] >> (pos % W)) & mask];
    }
```

```
    return s;
}

bool FromHex(const std::string& s, BigNum& a){
    if (s.size() >= 2 && s[0] == '0' && (s[1] == 'x' || s[1] == 'X'))
        return FromPow2String(s.substr(2), 4, a);
    return FromPow2String(s, 4, a);
}

std::string ToHex(const BigNum& a){
    return ToPow2String(a, 4);
}

bool FromBin(const std::string& s, BigNum& a){
    if (s.size() >= 2 && s[0] == '0' && (s[1] == 'b' || s[1] == 'B'))
        return FromPow2String(s.substr(2), 1, a);
    return FromPow2String(s, 1, a);
}

std::string ToBin(const BigNum& a){
    return ToPow2String(a, 1);
}


//lab2

void LongDiv(const BigNum& a, const BigNum& b, BigNum& q){
    BigNum dummy;
    LongDivMod(a,b,q,dummy);
}

BigNum LongGcd(const BigNum&a,const BigNum&b){
    if(BitLength(a)==0) return b;
    if(BitLength(b)==0) return a;
    BigNum a_ = a;
    BigNum b_ = b;
    int k =0;
    while(!(a_.d[0]&1)&&!(b_.d[0]&1)){
        a_ = LongShiftBitsToLow(a_,1);
        b_ = LongShiftBitsToLow(b_,1);
        k++;
    }
    while (!(a_.d[0]&1))
    {
        a_ = LongShiftBitsToLow(a_,1);
    }
    while (BitLength(b_)!=0)
    {
        while (!(b_.d[0]&1)){
            b_ = LongShiftBitsToLow(b_,1);
        }
        if(LongCmp(a_,b_)==1){
            std::swap(a_,b_);
        }
        LongSub(b_,a_,b_);

    }

    return LongShiftBitsToHigh(a_,k);
}

BigNum LongLcm(const BigNum& a, const BigNum &b){
    BigNum gcd = LongGcd(a,b);
    BigNum temp;
    LongDiv(a,gcd,temp);
    LongMul(temp, b, temp);
    return temp;
}

void LongAddMod(const BigNum& a, const BigNum& b, const BigNum& n,BigNum& c){
    LongAdd(a,b,c);
    if(LongCmp(c,n)>=0){
        LongSub(c,n,c);
    }
}

```

```
void LongSubMod(const BigNum& a, const BigNum& b, const BigNum& n,BigNum& c){
    if(LongSub(a,b,c)!=0){
        LongAdd(c,n,c);
    }
}

BigNum BarrettMu(const BigNum & n){
    int k = (BitLength(n) + W - 1) / W;
    BigNum beta2k = LongShiftDigitsToHigh(ToBigNum(1), 2 * k);
    BigNum mu;
    LongDiv(beta2k, n, mu);
    return mu;
}

BigNum BarrettReduction(const BigNum& a, const BigNum & n, const BigNum& mu){
    int k = (BitLength(n) + W - 1) / W;
    BigNum q = LongShiftDigitsToLow(a,k-1);
    LongMul(q,mu,q);
    q = LongShiftDigitsToLow(q,k+1);
    LongMul(q,n,q);
    LongSub(a,q,q);
    while(LongCmp(q,n)>=0){
        LongSub(q,n,q);
    }
    return q;

};

void LongMulMod(const BigNum& a,const BigNum& b, const BigNum& n, const BigNum& mu ,BigNum& c){
    LongMul(a,b,c);
    c = BarrettReduction(c, n, mu);
}

void LongSquareMod(const BigNum& a, const BigNum& n, const BigNum& mu,BigNum &c){
    LongSquare(a,c);
    c = BarrettReduction(c,n,mu);
}

void LongPowerMod(const BigNum& a, const BigNum& b, const BigNum& n, const BigNum& mu, BigNum& c){
    c = ToBigNum(1);
    BigNum a_ = BarrettReduction(a,n,mu);
    int m = BitLength(b);
    for(int i = 0; i < m;i++){
        if((b.d[i / W] >> (i % W)) & 1){
            LongMulMod(c,a_,n,mu,c);
        }
        LongSquareMod(a_,n,mu,a_);
    }
}
```

6.3 Файл tests.h

```
#pragma once

int runTests();
```

6.4 Файл tests.cpp

```
#include "tests.h"
#include "bignumarifmetic.h"
#include <cstdio>
#include <random>
#include <string>

static std::mt19937 rng(20261104);

// модуль для Barrett: k <= 31 цифри, тобто не більше 992 біт (див. BarrettMu)
const int MAX_MOD_BITS = (WORDS / 2 - 1) * W;

static BigNum RandomBigNum(int maxBits = BITS){
```

```
    int bits = 1 + rng() % maxBits;
    BigNum x = {};
    for (int i = 0; i < WORDS && i * W < bits; i++) x.d[i] = rng();
    if (bits % W != 0) x.d[bits / W] &= (1u << (bits % W)) - 1;
    return x;
}

// рівно bits біт (старший біт = 1)
static BigNum RandomExact(int bits){
    BigNum x = {};
    for (int i = 0; i < WORDS && i * W < bits; i++) x.d[i] = rng();
    if (bits % W != 0) x.d[bits / W] &= (1u << (bits % W)) - 1;
    x.d[(bits - 1) / W] |= 1u << ((bits - 1) % W);
    return x;
}

struct Modulus { BigNum n, mu; };

static Modulus MakeModulus(const BigNum& n){ return { n, BarrettMu(n) }; }
static Modulus RandomModulus(){ return MakeModulus(RandomExact(2 + rng() % (MAX_MOD_BITS - 1))); }

static BigNum Add(const BigNum& a, const BigNum& b){ BigNum c; LongAdd(a, b, c); return c; }
static BigNum Sub(const BigNum& a, const BigNum& b){ BigNum c; LongSub(a, b, c); return c; }
static BigNum Mul(const BigNum& a, const BigNum& b){ BigNum c; LongMul(a, b, c); return c; }
static BigNum Rem(const BigNum& a, const BigNum& n){ BigNum q, r; LongDivMod(a, n, q, r); return r; }
static BigNum Quot(const BigNum& a, const BigNum& n){ BigNum q; LongDiv(a, n, q); return q; }
static BigNum Hex(const std::string& s){ BigNum a; FromHex(s, a); return a; }
static bool   Eq(const BigNum& a, const BigNum& b){ return LongCmp(a, b) == 0; }
static bool   IsZero(const BigNum& a){ return BitLength(a) == 0; }

static BigNum AddM(const BigNum& a, const BigNum& b, const Modulus& m){ BigNum c; LongAddMod(a, b, m.n, c);
    return c; }
static BigNum SubM(const BigNum& a, const BigNum& b, const Modulus& m){ BigNum c; LongSubMod(a, b, m.n, c);
    return c; }
static BigNum MulM(const BigNum& a, const BigNum& b, const Modulus& m){ BigNum c; LongMulMod(a, b, m.n, m.mu
    , c); return c; }
static BigNum SqrM(const BigNum& a, const Modulus& m){ BigNum c; LongSquareMod(a, m.n, m.mu, c); return c; }
static BigNum PowM(const BigNum& a, const BigNum& e, const Modulus& m){ BigNum c; LongPowerMod(a, e, m.n, m.
    mu, c); return c; }

// випадкове число з [0, n)
static BigNum RandomBelow(const BigNum& n){ return Rem(RandomBigNum(BitLength(n)), n); }

static void Show(const char* name, const BigNum& x){
    printf("  %s = %s\n", name, ToHex(x).c_str());
}

static int failed = 0;

static void Report(const char* name, bool ok){
    printf("%s: %s\n", name, ok ? "Passed!" : "Failed!");
    if (!ok) failed++;
}

template <class F>
static bool ForRandom(int count, F check){
    for (int t = 0; t < count; t++)
        if (!check()) return false;
    return true;
}

const int RANDOM = 1000;

int runTests(){
    printf("--- Running Tests (%d bit numbers, modulus up to %d bit) ---\n\n", BITS, MAX_MOD_BITS);
    failed = 0;
    const Modulus M = MakeModulus(Hex("7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFED"));
    BigNum a = Hex("6C1A4F0E3B2D59877A13C4E2F05B6D18A9C3E7F2014B5D6E8F90A1B2C3D4E5F6");
    BigNum b = Hex("3F2E1D0C9B8A79685746352413F2E1D0CFBEAD9C8B7A69584736251403F2E1D0");
    BigNum c = Hex("1234567890ABCDEF1234567890ABCDEF");
    Show("n ", M.n); Show("mu", M.mu);
    Show("a ", a); Show("b ", b); Show("c ", c);
    printf("\n");

    BigNum lhs = MulM(AddM(a, b, M), c, M);
    Show("(a+b)*c mod n  ", lhs);
```

```
    Show("c*(a+b) mod n  ", MulM(c, AddM(a, b, M), M));
    Show("a*c + b*c mod n", AddM(MulM(a, c, M), MulM(b, c, M), M));
    Report("Test 1 [(a+b)*c == c*(a+b) == a*c + b*c (mod n)]",
           Eq(lhs, Hex("CDCCA09184A1B5B1B90E37039BC68063AC88DFE8BB795659305430D1557A619")) &&
           Eq(lhs, MulM(c, AddM(a, b, M), M)) && Eq(lhs, AddM(MulM(a, c, M), MulM(b, c, M), M)) &&
           ForRandom(RANDOM, []{
               Modulus m = RandomModulus();
               BigNum x = RandomBelow(m.n), y = RandomBelow(m.n), z = RandomBelow(m.n);
               BigNum l = MulM(AddM(x, y, m), z, m);
               return Eq(l, MulM(z, AddM(x, y, m), m)) && Eq(l, AddM(MulM(x, z, m), MulM(y, z, m), m));
           }));

    BigNum sum = {};
    for (int k = 0; k < 100; k++) sum = AddM(sum, a, M);
    BigNum na = MulM(ToBigNum(100), a, M);
    Show("100*a mod n      ", na);
    Show("a + ... + a mod n", sum);
    Report("Test 2 [k*a == a+a+...+a (mod n), k = 100..1000]",
           Eq(na, sum) && Eq(na, Hex("3A46E18F1DB6F8EBAFB8E8A5E3B69DA250869A8881707F30147F29D47F29DA54")) &&
           ForRandom(RANDOM, []{
               Modulus m = RandomModulus();
               BigNum x = RandomBelow(m.n), s = {};
               uint32_t k = 100 + rng() % 901;
               for (uint32_t t = 0; t < k; t++) s = AddM(s, x, m);
               return Eq(MulM(Rem(ToBigNum(k), m.n), x, m), s);
           }));

    BigNum fermat = PowM(a, Sub(M.n, ToBigNum(1)), M);
    Show("a^(n-1) mod n, n = 2^255-19", fermat);
    Report("Test 3 [a^phi(n) == 1 (mod n), n prime: 2^255-19, Mersenne 2^127-1, 2^521-1, 2^607-1]",
           Eq(fermat, ToBigNum(1)) &&
           ForRandom(30, []{
               static const int P[] = { 127, 521, 607 };
               int p = P[rng() % 3];
               Modulus m = MakeModulus(Sub(LongShiftBitsToHigh(ToBigNum(1), p), ToBigNum(1)));
               BigNum x = RandomBelow(m.n);
               if (IsZero(x)) x = ToBigNum(1);
               return Eq(PowM(x, Sub(m.n, ToBigNum(1)), m), ToBigNum(1));
           }));

    BigNum p3; LongPower(ToBigNum(3), ToBigNum(100), p3);
    Modulus M3 = MakeModulus(p3);
    BigNum phi3 = Mul(ToBigNum(2), Quot(p3, ToBigNum(3)));
    BigNum a1 = Rem(Add(a, ToBigNum(1)), p3);
    BigNum euler3 = PowM(a1, phi3, M3);
    Show("n = 3^100             ", p3);
    Show("(a+1)^phi(n) mod 3^100", euler3);
    Report("Test 4 [a^(2*3^(k-1)) == 1 (mod 3^k), a not divisible by 3]",
           Eq(euler3, ToBigNum(1)) &&
           ForRandom(30, []{
               int k = 1 + rng() % 620;                  // 3^620 < 2^992
               BigNum n3, x; LongPower(ToBigNum(3), ToBigNum(k), n3);
               Modulus m = MakeModulus(n3);
               BigNum phi = Mul(ToBigNum(2), Quot(n3, ToBigNum(3)));
               do x = RandomBelow(n3); while (IsZero(Rem(x, ToBigNum(3))));
               return Eq(PowM(x, phi, m), ToBigNum(1));
           }));
    BigNum x = Mul(Mul(a, b), Rem(c, M.n));
    x = Rem(Mul(x, a), Mul(M.n, M.n));                   // x < n^2
    Show("x               ", x);
    Show("Barrett(x)      ", BarrettReduction(x, M.n, M.mu));
    Show("x mod n (DivMod)", Rem(x, M.n));
    Report("Test 5 [BarrettReduction(x) == x mod n, x < n^2]",
           Eq(BarrettReduction(x, M.n, M.mu), Rem(x, M.n)) &&
           Eq(BarrettReduction(Sub(Mul(M.n, M.n), ToBigNum(1)), M.n, M.mu), Sub(M.n, ToBigNum(1))) &&
           Eq(BarrettReduction(M.n, M.n, M.mu), ToBigNum(0)) &&
           ForRandom(RANDOM, []{
               Modulus m = RandomModulus();
               BigNum y = Rem(RandomBigNum(2 * BitLength(m.n)), Mul(m.n, m.n));
               return Eq(BarrettReduction(y, m.n, m.mu), Rem(y, m.n));
           }));

    BigNum g = LongGcd(a, b), l = LongLcm(a, b);
    Show("gcd(a, b)", g); Show("lcm(a, b)", l);
    Report("Test 6 [a, b divisible by gcd, gcd(a/g, b/g) == 1, gcd * lcm == a * b]",
           Eq(g, ToBigNum(6)) &&
```

```
            Eq(l, Hex("47253132
        B2A40A502D805AE83B8FF272E9D0FE2264B1CE6B9D44CFE9FC3728E360F348133AB5F8E368E37EE8AF11C7687455B4EC0A9B60640C7706C154602
        ")) &&
            Eq(LongGcd(a, ToBigNum(0)), a) && Eq(LongGcd(ToBigNum(0), a), a) && Eq(LongGcd(a, a), a) &&
            Eq(LongGcd(a, ToBigNum(1)), ToBigNum(1)) && Eq(LongLcm(a, ToBigNum(0)), ToBigNum(0)) &&
            ForRandom(RANDOM, []{
                BigNum common = RandomBigNum(200);
                if (IsZero(common)) common = ToBigNum(1);
                BigNum u = Mul(RandomBigNum(400), common), v = Mul(RandomBigNum(400), common);
                if (IsZero(u) || IsZero(v)) return true;
                BigNum gg = LongGcd(u, v);
                return IsZero(Rem(u, gg)) && IsZero(Rem(v, gg)) && IsZero(Rem(gg, common)) &&
                       Eq(LongGcd(Quot(u, gg), Quot(v, gg)), ToBigNum(1)) &&
                       Eq(Mul(gg, LongLcm(u, v)), Mul(u, v));
            }));

    Show("a + b mod n", AddM(a, b, M));
    Show("a - b mod n", SubM(a, b, M));
    Show("b - a mod n", SubM(b, a, M));
    Report("Test 7 [(a+b)-b == a, (a-b)+b == a, a-a == 0 (mod n), results < n]",
            Eq(AddM(a, b, M), Hex("2B486C1AD6B7D2EFD159FA07044E4EE97982958E8CC5C6C6D6C6C6C6C7C7C7D9")) &&
            Eq(SubM(a, b, M), Hex("2CEC32019FA2E01F22CD8FBEDC688B47DA053A5575D0F416485A7C9EBFE20426")) &&
            Eq(SubM(b, a, M), Hex("5313CDFE605D1FE0DD327041239774B825FAC5AA8A2F0BE9B7A58361401DFBC7")) &&
            ForRandom(RANDOM, []{
                Modulus m = RandomModulus();
                BigNum u = RandomBelow(m.n), v = RandomBelow(m.n);
                BigNum s = AddM(u, v, m), d = SubM(u, v, m);
                return LongCmp(s, m.n) < 0 && LongCmp(d, m.n) < 0 &&
                       Eq(SubM(s, v, m), u) && Eq(AddM(d, v, m), u) && IsZero(SubM(u, u, m)) &&
                       Eq(s, Rem(Add(u, v), m.n));
            }));

    Show("a * b mod n", MulM(a, b, M));
    Show("a^2 mod n  ", SqrM(a, M));
    Show("a^b mod n  ", PowM(a, b, M));
    Report("Test 8 [a*b mod n == (a*b) % n, a^2 == a*a (mod n), a^2 == a*a]",
            Eq(MulM(a, b, M), Hex("3A2D3819A7A5D04BCFBA5F049269E9ACAF8249429EE001A305C1354EC9B61516")) &&
            Eq(SqrM(a, M), Hex("48A031F31C028489B8944CE4F0A3CE196D47FC161F1867E73DEE4227380B1A41")) &&
            ForRandom(RANDOM, []{
                Modulus m = RandomModulus();
                BigNum u = RandomBelow(m.n), v = RandomBelow(m.n), sq;
                LongSquare(u, sq);
                return Eq(MulM(u, v, m), Rem(Mul(u, v), m.n)) && Eq(SqrM(u, m), MulM(u, u, m)) &&
                       Eq(sq, Mul(u, u));
            }));

    Report("Test 9 [a^(e+f) == a^e * a^f, a^(e*f) == (a^e)^f (mod n), a^0 == 1, a^1 == a]",
            Eq(PowM(a, b, M), Hex("41FA9242A62F71FCBD59F3368FFA8400A445268C9A885C66B47D8ECEDBD20AD8")) &&
            Eq(PowM(a, ToBigNum(0), M), ToBigNum(1)) && Eq(PowM(a, ToBigNum(1), M), a) &&
            ForRandom(100, []{
                Modulus m = RandomModulus();
                BigNum u = RandomBelow(m.n), e = RandomBigNum(400), f = RandomBigNum(400), small =
     RandomBigNum(32);
                BigNum ue = PowM(u, e, m);
                return Eq(PowM(u, Add(e, f), m), MulM(ue, PowM(u, f, m), m)) &&
                       Eq(PowM(u, Mul(e, small), m), PowM(ue, small, m));
            }));

    printf("\n%s (failed: %d)\n----------------------------\n\n",
            failed ? "SOME TESTS FAILED" : "All tests passed", failed);
    return failed;
}
```

6.5 Файл benchmarks.h

```
#pragma once

void runBenchmarks();
```

6.6 Файл benchmarks.cpp

```cpp
#include "benchmarks.h"
#include "bignumarifmetic.h"
#include <cstdio>
#include <chrono>
#include <random>
#include <string>
#include <intrin.h>

static std::mt19937 rng(20261104);

static BigNum RandomBigNum(int bits){
    BigNum x = {};
    for (int i = 0; i < WORDS && i * W < bits; i++) x.d[i] = rng();
    if (bits % W != 0) x.d[bits / W] &= (1u << (bits % W)) - 1;
    return x;
}

static BigNum RandomExact(int bits){
    BigNum x = RandomBigNum(bits);
    x.d[(bits - 1) / W] |= 1u << ((bits - 1) % W);
    return x;
}

static BigNum Rem(const BigNum& a, const BigNum& n){ BigNum q, r; LongDivMod(a, n, q, r); return r; }

const int POOL = 256;
const int P = POOL - 1;
const int D = WORDS - 1;

struct ModSet {
    int bits;
    BigNum n[POOL], mu[POOL];
    BigNum a[POOL], b[POOL];
    BigNum x[POOL];
    BigNum g1[POOL], g2[POOL];

    void Init(int bitsN){
        bits = bitsN;
        for (int i = 0; i < POOL; i++){
            n[i] = RandomExact(bits);
            mu[i] = BarrettMu(n[i]);
            a[i] = Rem(RandomBigNum(bits), n[i]);
            b[i] = Rem(RandomBigNum(bits), n[i]);
            LongMul(a[i], b[i], x[i]);
            g1[i] = RandomExact(bits);
            g2[i] = RandomExact(bits);
        }
    }
};

static ModSet S256, S512, S992;

static volatile uint32_t sink;

template <class F>
static void Bench(const char* name, int iterations, F op){
    for (int i = 0; i < iterations / 10 + 1; i++) op(i);
    auto start = std::chrono::steady_clock::now();
    uint64_t c0 = __rdtsc();
    for (int i = 0; i < iterations; i++) op(i);
    uint64_t c1 = __rdtsc();
    auto end = std::chrono::steady_clock::now();
    double ns = std::chrono::duration<double, std::nano>(end - start).count() / iterations;
    printf("| %-36s | %10d | %14.1f | %14.0f |\n", name, iterations, ns, (double)(c1 - c0) / iterations);
}

static void Header(const char* title){
    printf("\n%s\n", title);
    printf("| %-36s | %10s | %14s | %14s |\n", "Operation", "Iterations", "ns / op", "cycles / op");
    printf("|--------------------------------------|------------|----------------|----------------|\n");
}

static void BenchSet(ModSet& S, int scale){
    char name[64];
    #define NAME(op) (snprintf(name, sizeof name, "%s (%d bit)", op, S.bits), name)

```

```
    Bench(NAME("LongGcd"), 2000 * scale, [&](int i){
        sink = sink ^ LongGcd(S.g1[i & P], S.g2[i & P]).d[0]; });
    Bench(NAME("LongLcm"), 300 * scale, [&](int i){
        sink = sink ^ LongLcm(S.g1[i & P], S.g2[i & P]).d[i & D]; });
    Bench(NAME("LongAddMod"), 1000000, [&](int i){
        BigNum c; LongAddMod(S.a[i & P], S.b[i & P], S.n[i & P], c); sink = sink ^ c.d[0]; });
    Bench(NAME("LongSubMod"), 1000000, [&](int i){
        BigNum c; LongSubMod(S.a[i & P], S.b[i & P], S.n[i & P], c); sink = sink ^ c.d[0]; });
    Bench(NAME("BarrettMu"), 300 * scale, [&](int i){
        sink = sink ^ BarrettMu(S.n[i & P]).d[0]; });
    Bench(NAME("BarrettReduction"), 50000, [&](int i){
        sink = sink ^ BarrettReduction(S.x[i & P], S.n[i & P], S.mu[i & P]).d[0]; });
    Bench(NAME("x mod n via LongDivMod"), 300 * scale, [&](int i){
        sink = sink ^ Rem(S.x[i & P], S.n[i & P]).d[0]; });
    Bench(NAME("LongMulMod"), 50000, [&](int i){
        BigNum c; LongMulMod(S.a[i & P], S.b[i & P], S.n[i & P], S.mu[i & P], c); sink = sink ^ c.d[0]; });
    Bench(NAME("LongPowerMod"), 20 * scale, [&](int i){
        BigNum c; LongPowerMod(S.a[i & P], S.b[i & P], S.n[i & P], S.mu[i & P], c); sink = sink ^ c.d[0]; })
     ;

    #undef NAME
}

static void BenchSquare(ModSet& S){
    char name[64];
    #define NAME(op) (snprintf(name, sizeof name, "%s (%d bit)", op, S.bits), name)

    Bench(NAME("LongMul(a, a)"), 100000, [&](int i){
        BigNum c; LongMul(S.a[i & P], S.a[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench(NAME("LongSquare(a)"), 100000, [&](int i){
        BigNum c; LongSquare(S.a[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench(NAME("LongMulMod(a, a)"), 50000, [&](int i){
        BigNum c; LongMulMod(S.a[i & P], S.a[i & P], S.n[i & P], S.mu[i & P], c); sink = sink ^ c.d[0]; });
    Bench(NAME("LongSquareMod(a)"), 50000, [&](int i){
        BigNum c; LongSquareMod(S.a[i & P], S.n[i & P], S.mu[i & P], c); sink = sink ^ c.d[0]; });

    #undef NAME
}

void runBenchmarks(){
    printf("--- Running Benchmarks (%d bit numbers, %d random moduli per size) ---\n", BITS, POOL);

    S256.Init(256);
    S512.Init(512);
    S992.Init((WORDS / 2 - 1) * W);

    Header("Modular arithmetic");
    BenchSet(S256, 8);
    BenchSet(S512, 3);
    BenchSet(S992, 1);

    Header("Variant 1: squaring vs multiplication");
    BenchSquare(S256);
    BenchSquare(S512);
    BenchSquare(S992);

    printf("\nLongPowerMod: exponent has the same bit length as the modulus\n");
    printf("cycles = TSC (__rdtsc), counts at the nominal CPU frequency\n");
    printf("------------------------------\n\n");
}
```

6.7 Файл main.cpp

```
#include "tests.h"
#include "benchmarks.h"
#include "bignumarifmetic.h"
#define NOMINMAX
#include <windows.h>
#include <cstdio>

int main(){
    SetConsoleOutputCP(CP_UTF8);

```

```
11
12      BigNum A, B, N;
13      FromHex("0
         xcebe60efd949039fa8d8bac7d3e17db03f3e060531a329396ff9f9ab618600487e6efa1059fa75c882f01ab7d0564b58030dfdf6846db25d7e02
         ", A);
14      FromHex("0
         xd022e8f4fa56820673ae38cc8820c81662b720857e4fb620c7fcc6e3869e9435d24fe2b17f7a0769fcf5088580f8f7388ce84f8151cf034faeb7
         ", B);
15      FromHex("0
         xdcb6124729054a97aee4a7edffc78672de847485fb6db32472c1b47f0741328ce0469fa2602985b97518b05112d9d65ff664f742b16dbb7c9e86
         ", N);
16
17      printf("A = %s\n", ToHex(A).c_str());
18      printf("B = %s\n", ToHex(B).c_str());
19      printf("N = %s\n\n", ToHex(N).c_str());
20
21      printf("gcd(A, B) = %s\n\n", ToHex(LongGcd(A, B)).c_str());
22      printf("lcm(A, B) = %s\n\n", ToHex(LongLcm(A, B)).c_str());
23
24      BigNum q;
25      LongDivMod(A, N, q, A);
26      LongDivMod(B, N, q, B);
27
28      BigNum mu = BarrettMu(N);
29      BigNum sum, diff, prod, pw;
30      LongAddMod(A, B, N, sum);
31      LongSubMod(A, B, N, diff);
32      LongMulMod(A, B, N, mu, prod);
33      LongPowerMod(A, B, N, mu, pw);
34
35      printf("A + B mod N = %s\n\n", ToHex(sum).c_str());
36      printf("A - B mod N = %s\n\n", ToHex(diff).c_str());
37      printf("A * B mod N = %s\n\n", ToHex(prod).c_str());
38      printf("A ^ B mod N = %s\n\n", ToHex(pw).c_str());
39
40      //int failed = runTests();
41      //runBenchmarks();
42     //return failed;
43  }
```